



**KARADENİZ
TEKNİK ÜNİVERSİTESİ**
KARADENİZ TECHNICAL UNIVERSITY
1955

PARALEL PROGRAMLAMA MPI İLE PARALEL MATRİS ÇARPIMI VE ÇOKLU DİL ANALİZİ

Grup Üyeleri

413794-Ali Eren Temel

425474-Halil İbrahim Çakır

414783-Hasan Furkan Mavuş

Ders

Paralel Bilgisayarlar

Öğretim Üyesi

Öğr. Gör. Dr. ZAFER YAVUZ

İÇİNDEKİLER

1. Giriş
2. Kullanılan Teknolojiler
3. Sistem Özellikleri
4. Paralel Matris Çarpımı Yaklaşımı
5. Performans Ölçütleri
6. Deney Sonuçları
7. Sonuçların Değerlendirilmesi
8. Sonuç

1. Giriş

Bu projede MPI (Message Passing Interface) kullanılarak paralel matris-matris çarpımı gerçekleştirilmiştir. Çalışmanın amacı, paralel programlamanın performans üzerindeki etkisini incelemek ve farklı süreç sayılarında elde edilen hızlanma değerlerini analiz etmektir.

Projede hem C dili hem de Python dili kullanılarak iki ayrı paralel uygulama geliştirilmiştir. Python tarafında mpi4py kütüphanesi kullanılmıştır.

Matrisler dosyalardan okunmuş ve MPI kolektif haberleşme işlemleri yardımıyla süreçler arasında paylaştırılmıştır. Deneyler küçük ve büyük ölçekli problemler üzerinde gerçekleştirilmiştir.

Kullanılan matris boyutları:

- $N = 16$
- $N = 256$

Bu sayede paralel programlamanın küçük ve büyük veri boyutlarındaki davranışı gözlemlenmiştir.

2. Kullanılan Teknolojiler

Projede aşağıdaki teknolojiler kullanılmıştır:

Teknoloji	Açıklama
-----------	----------

C	Yüksek performanslı MPI uygulaması
---	------------------------------------

Python	mpi4py ile paralel uygulama
--------	-----------------------------

MPI	Süreçler arası haberleşme
-----	---------------------------

OpenMPI	MPI çalışma ortamı
---------	--------------------

NumPy	Python matris işlemleri
-------	-------------------------

WSL / Linux	Çalışma ortamı
-------------	----------------

3. Sistem Özellikleri

Deneyler üç farklı bilgisayar ortamında gerçekleştirilmiştir.

Sistem 1 (Halil İbrahim)

Özellik	Değer
İşlemci	Intel Core i5 10th Gen
Çekirdek Sayısı	8
İşletim Sistemi	Linux (WSL)

Sistem 2 (Ali Eren)

Özellik	Değer
İşlemci	Intel Core i5 12th Gen
Çekirdek Sayısı	12
İşletim Sistemi	Linux (WSL)

Sistem 3 (Hasan Furkan)

Özellik	Değer
İşlemci	Intel Core i5 14th Gen
Çekirdek Sayısı	14
İşletim Sistemi	Linux(WSL)

4. Paralel Matris Çarpımı Yaklaşımı

Projede kare matrisler için aşağıdaki işlem gerçekleştirilmiştir:

$$C = A \times B$$

Programın çalışma adımları aşağıdaki gibidir:

1. Ana süreç matrisleri dosyadan okur.
2. A matrisinin satırları MPI Scatter ile süreçlere dağıtılır.
3. B matrisi MPI Bcast ile tüm süreçlere gönderilir.
4. Her süreç kendisine ait satırlar üzerinde hesaplama yapar.
5. Sonuçlar MPI Gather ile ana süreçte birleştirilir.

Bu yöntem sayesinde işlem yükü süreçler arasında paylaştırılmıştır.

5. Performans Ölçütleri

5.1 Çalışma Süresi

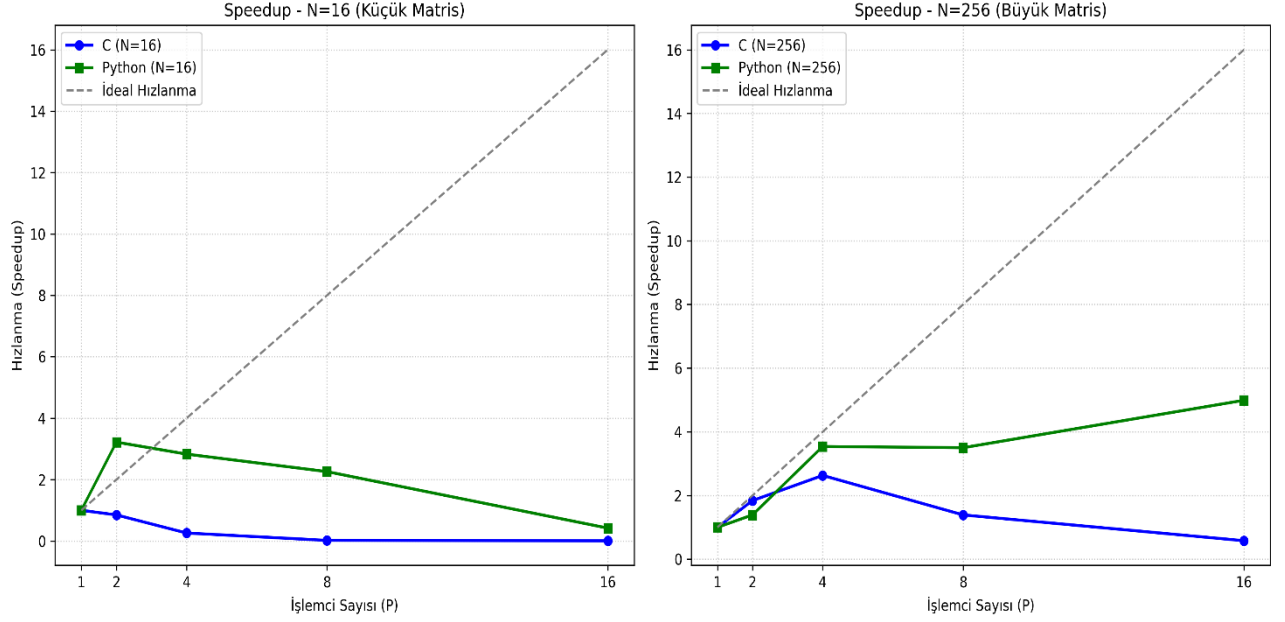
$T_p(P)$, programın P adet süreç ile çalıştırılması sonucu ölçülen toplam çalışma süresidir.

5.2 Speedup

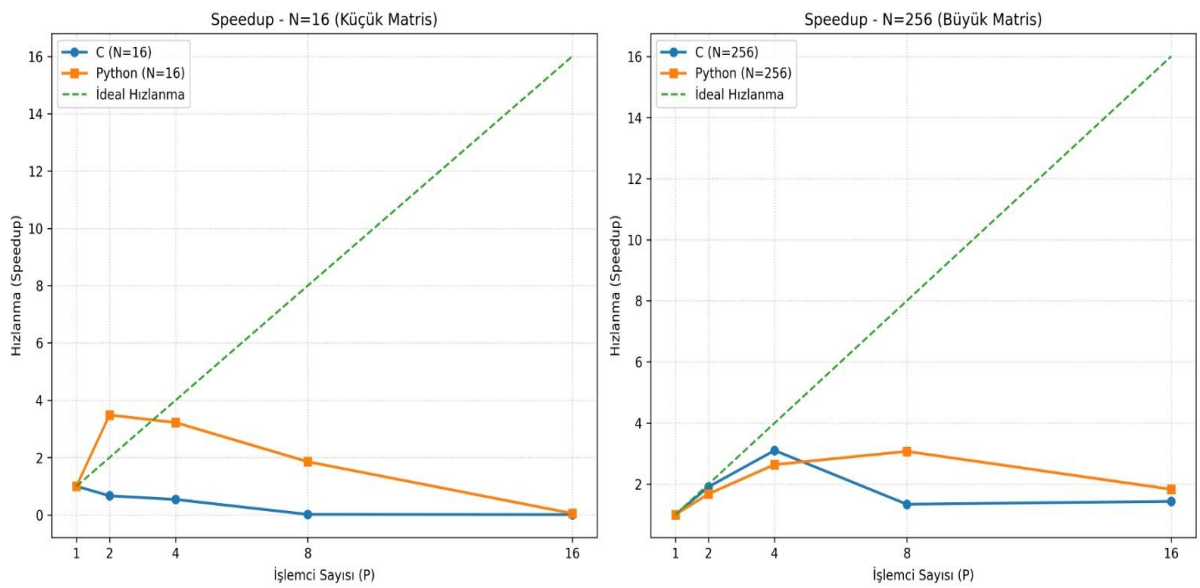
$$S(P) = \frac{T_p(1)}{T_p(P)}$$

Bu değer paralel programın tek süreçli çalışmaya göre ne kadar hızlandığını göstermektedir.

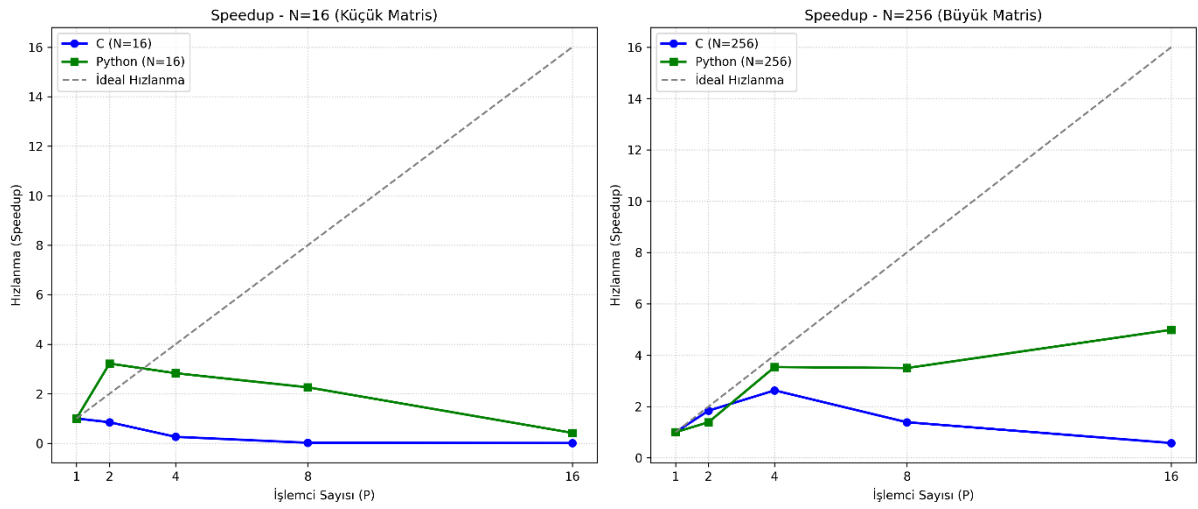
Ali Eren Bilgisayarı sonuçları(i5 12th 12 çekirdek)



Halil İbrahim Bilgisayarı(i5 10th 8 çekirdek)



Hasan Furkan Bilgisayarı(i5 13th 14 çekirdek)

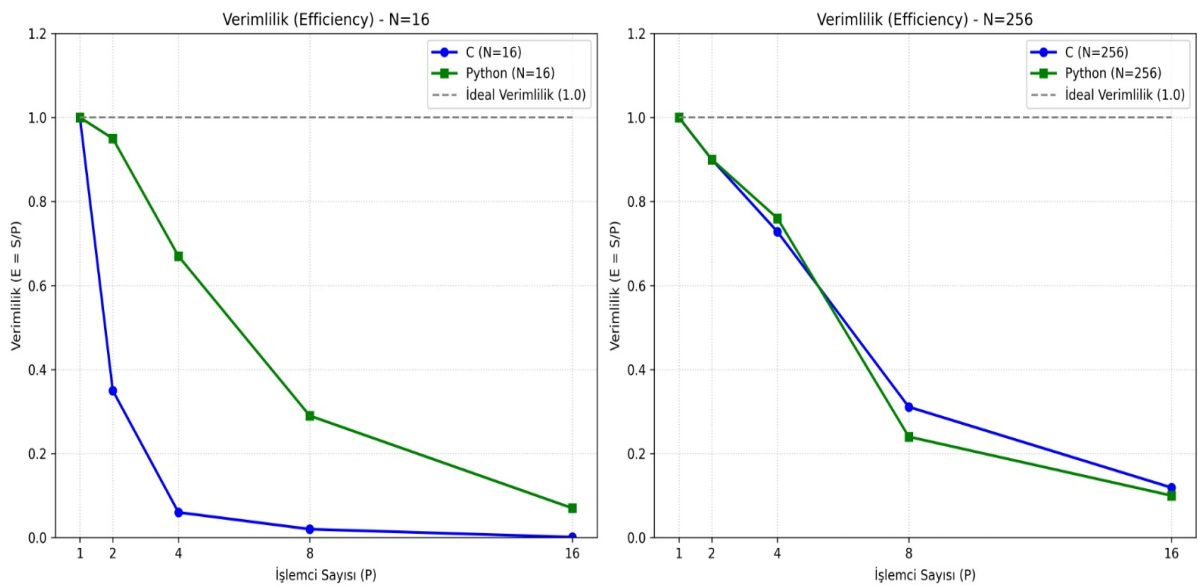


5.3 Efficiency

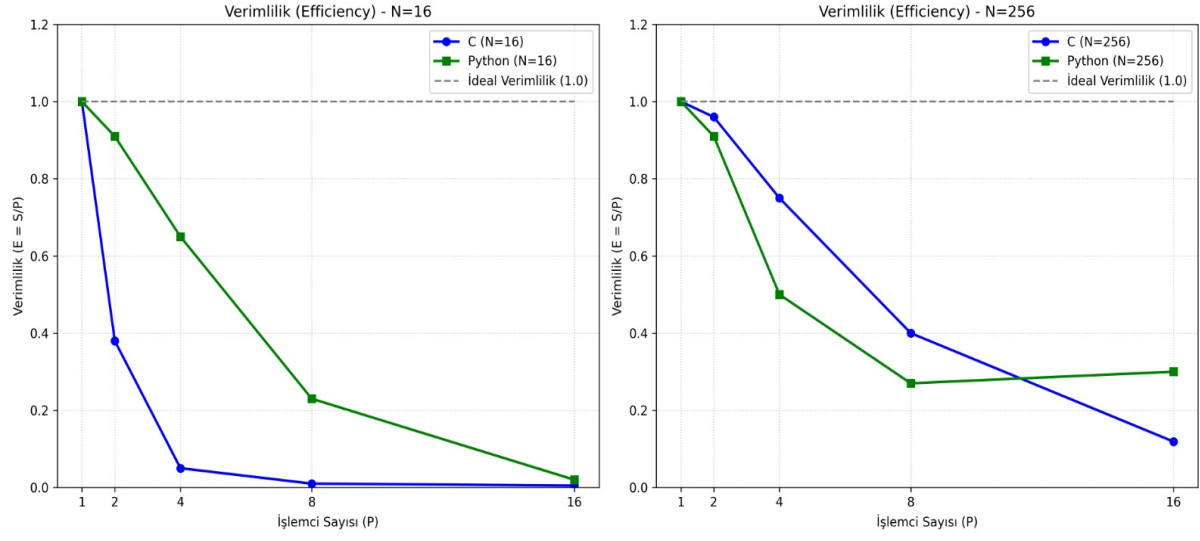
$$E(P) = \frac{S(P)}{P}$$

Efficiency değeri süreçlerin ne kadar verimli kullanıldığını göstermektedir.

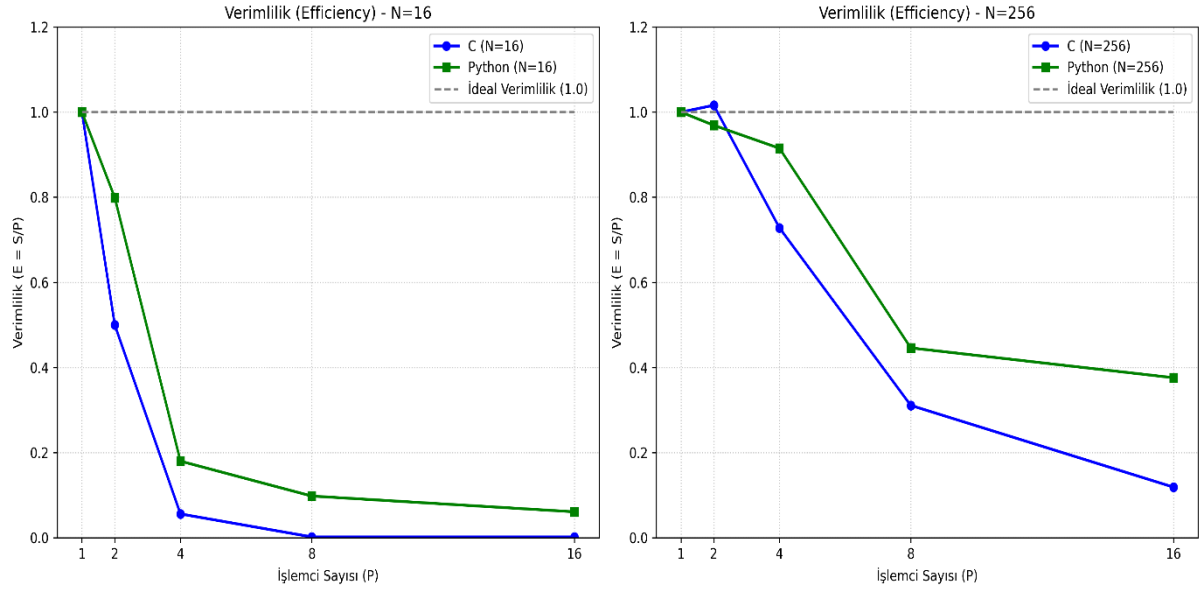
Ali Eren Bilgisayarı(i5 12th 12 Çekirdek)



Halil İbrahim Bilgisayarı(i5 10th 8 çekirdek)



Hasan Furkan Bilgisayarı(i5 13th 14 çekirdek)



6. Deney Sonuçları

Sistem 1 — Intel Core i5 10th Gen (8 Çekirdek)

N = 16

Süreç Sayısı (P)	C_Süre (sn)	C_Speedup	C_Verim (%)	Py_Süre (sn)	Py_Speedup	Py_Verim (%)
1	0,000010	1,000	100	0,001385	1,000	100
2	0,000013	0,769	38,46	0,001521	0,910	45,50
4	0,000042	0,238	5,95	0,002114	0,655	16,38
8	0,000132	0,076	0,95	0,005845	0,237	2,96
16	0,000151	0,066	0,41	0,073157	0,019	0,12

N = 256

Süreç Sayısı (P)	C_Süre (sn)	C_Speedup	C_Verim (%)	Py_Süre (sn)	Py_Speedup	Py_Verim (%)
1	0,024394	1,000	100	0,089594	1,000	100
2	0,012649	1,928	96,42	0,049173	1,822	91,10
4	0,008076	3,020	75,50	0,044079	2,032	50,80
8	0,007587	3,215	40,19	0,040649	2,204	27,55
16	0,013551	1,800	11,25	0,153781	0,583	3,64

Sistem 2 — Intel Core i5 12th Gen (12 Çekirdek)

N = 16

Süreç Sayısı (P)	C_Süre (sn)	C_Speedup	C_Verim (%)	Py_Süre (sn)	Py_Speedup	Py_Verim (%)
1	0,000010	1,000	100	0,004849	1,000	100
2	0,000014	0,714	35,70	0,002532	1,915	95,75
4	0,000036	0,278	6,95	0,001805	2,687	67,18
8	0,000053	0,189	2,36	0,002033	2,385	29,81
16	0,000375	0,027	0,17	0,004314	1,124	7,02

N = 256

Süreç Sayısı (P)	C_Süre (sn)	C_Speedup	C_Verim (%)	Py_Süre (sn)	Py_Speedup	Py_Verim (%)
1	0,009589	1,000	100	0,071303	1,000	100
2	0,005277	1,817	90,85	0,039223	1,818	90,90
4	0,003090	3,103	77,58	0,023310	3,059	76,48
8	0,004887	1,962	24,53	0,035860	1,988	24,85
16	0,005259	1,823	11,39	0,041366	1,724	10,78

Sistem 3 — 14 Çekirdekli Sistem

N = 16

Süreç Sayısı (P)	C_Süre (sn)	C_Speedup	C_Verim (%)	Py_Süre (sn)	Py_Speedup	Py_Verim (%)
1	0,000010	1,000	100	0,001919	1,000	100
2	0,000010	1,000	50,00	0,001201	1,598	79,90
4	0,000045	0,222	5,56	0,002670	0,719	17,97
8	0,000529	0,019	0,24	0,002456	0,781	9,76
16	0,000307	0,033	0,20	0,001965	0,977	6,11

N = 256

Süreç Sayısı (P)	C_Süre (sn)	C_Speedup	C_Verim (%)	Py_Süre (sn)	Py_Speedup	Py_Verim (%)
1	0,011366	1,000	100	0,930756	1,000	100
2	0,005594	2,032	101,60	0,480289	1,938	96,90
4	0,003904	2,912	72,80	0,254275	3,661	91,53
8	0,004568	2,488	31,10	0,260731	3,570	44,63
16	0,005987	1,898	11,86	0,154815	6,012	37,57

Üç Farklı Sistemde MPI Performans Karşılaştırması Raporu

Bu çalışmada üç farklı işlemci mimarisinin (8, 12 ve 14 çekirdekli) C ve Python dillerindeki MPI (Message Passing Interface) paralel hesaplama performansları karşılaştırılmıştır. Elde edilen temel bulgular şunlardır:

- **Düşük Veri Yükünde İletişim Maliyeti (N=16):** Her üç sistemde de küçük matrislerde paralel işlem yapmak performansı düşürmüştür. İşlemleri çekirdeklere dağıtma süresi (iletişim maliyeti), hesaplama süresini aştığı için küçük verilerde tek çekirdek kullanımı daha verimlidir.
- **Sistem 1 (8 Çekirdek - i5 10. Nesil):** Fiziksel çekirdek sınırına (P=8) kadar başarılı bir şekilde ölçeklenmiş (C'de 3.21x hızlanma) ancak P=16 denemesinde donanım limiti aşıldığı için "bağlam değişimi (context switching)" yükü oluşmuş ve performans ciddi şekilde düşmüştür (Verim: %11.25).
- **Sistem 2 (12 Çekirdek - i5 12. Nesil):** Güçlü IPC (tek çekirdek) performansı sayesinde C dilindeki en hızlı mutlak çalışma süresini (0.009 sn) bu sistem vermiştir. Fakat sahip olduğu hibrit (Performans/Verimlilik) çekirdek yapısının MPI ile tam optimize olmaması nedeniyle P=4'ten sonraki paralel ölçeklenmesi zayıf kalmıştır.
- **Sistem 3 (14 Çekirdek):** En geniş fiziksel çekirdek havuzuna sahip olan bu sistem, en iyi paralel ölçeklenmeyi sunmuştur. P=16'da bile darboğaza girmeden çalışmış ve Python implementasyonunda testin en yüksek göreceli hızlanma değerine (6.01x) ulaşmıştır. Ayrıca C'de P=2 için "Superlinear Speedup" (%101.6 verim) gözlemlenmiştir.

Genel Değerlendirme: Derlenen bir dil olan C, her üç sistemde de Python'dan mutlak süre olarak ortalama yüz kat daha hızlı çalışmıştır. Testler, başarılı bir paralelleştirme için sistemdeki fiziksel çekirdek sayısının aşılmaması ve veri boyutunun iletişim maliyetini kurtaracak kadar büyük olması gerektiğini kanıtlamıştır. En yüksek tekli işlem gücü Sistem 2'de, en iyi paralel donanım ölçeklenmesi ise Sistem 3'te görülmüştür.

7. Sonuçların Değerlendirilmesi

Deney sonuçları incelendiğinde aşağıdaki sonuçlar elde edilmiştir:

- Büyük boyutlu matrislerde paralel programlama daha başarılı sonuçlar vermektedir.
- Küçük ve orta ölçekli problemlerde MPI haberleşme maliyetleri performansı etkileyebilmektedir.
- C dili Python'a göre genel olarak daha düşük çalışma süresi sağlamıştır.
- Python uygulamaları geliştirme kolaylığı sunmasına rağmen yorumlayıcı maliyetlerinden etkilenmektedir.
- Süreç sayısı arttıkça belirli bir seviyeye kadar speedup artışı gözlemlenmiştir.
- Süreç sayısının çekirdek sayısına yaklaşması veya aşması durumunda efficiency değerleri düşmeye başlamıştır.
- 12 ve 14 çekirdekli sistemlerde büyük matrislerde daha başarılı paralel performans elde edilmiştir.
- Donanım özellikleri MPI performansını doğrudan etkilemektedir.

Özellikle $N = 256$ boyutundaki matrislerde elde edilen yüksek speedup değerleri MPI tabanlı paralel programlamanın büyük problemlerde etkili olduğunu göstermektedir.

8. Sonuç

Bu projede MPI kullanılarak paralel matris–matris çarpımı başarıyla gerçekleştirilmiştir. Yapılan deneyler sonucunda paralel programlamanın özellikle büyük boyutlu problemlerde önemli performans avantajı sağladığı görülmüştür.

Farklı sistemlerde yapılan deneyler donanım özelliklerinin MPI performansı üzerinde doğrudan etkili olduğunu göstermiştir. Özellikle çekirdek sayısı arttıkça paralel performans belirli seviyeye kadar iyileşmiştir.

Sonuç olarak MPI, yüksek hesaplama gücü gerektiren problemlerde etkili ve verimli bir paralel programlama çözümü sunmaktadır.